

[On This Page](#) >

Advanced Vars Examples

This guide demonstrates complex, real-world patterns for the vars system.

Certificate Authority with Intermediate Certificates

This example shows how to create a complete certificate authority with root and intermediate certificates using dependencies.

```
1 {
2   # Generate root CA (not deployed to machines)
3   clan.core.vars.generators.root-ca = {
4     files."ca.key" = {
5       secret = true;
6       deploy = false; # Keep root key offline
7     };
8     files."ca.crt".secret = false;
9     runtimeInputs = [ pkgs.step-cli ];
10    script = ''
11      step certificate create "My Root CA" \
12        $out/ca.crt $out/ca.key \
13        --profile root-ca \
14        --no-password \
15        --insecure \
16        --not-after 87600h
17    '';
18  };
19
20  # Generate intermediate key
21  clan.core.vars.generators.intermediate-key = {
22    files."intermediate.key" = {
23      secret = true;
```

```

24     deploy = true;
25 };
26 runtimeInputs = [ pkgs.step-cli ];
27 script = ''
28     step crypto keypair \
29         $out/intermediate.pub \
30         $out/intermediate.key \
31         --no-password \
32         --insecure
33 '';
34 };
35
36 # Generate intermediate certificate signed by root
37 clan.core.vars.generators.intermediate-cert = {
38     files."intermediate.crt".secret = false;
39     dependencies = [
40         "root-ca"
41         "intermediate-key"
42     ];
43     runtimeInputs = [ pkgs.step-cli ];
44     script = ''
45         step certificate create "My Intermediate CA" \
46             $out/intermediate.crt \
47             --key $in/intermediate-key/intermediate.key \
48             --ca $in/root-ca/ca.crt \
49             --ca-key $in/root-ca/ca.key \
50             --profile intermediate-ca \
51             --not-after 8760h \
52             --no-password \
53             --insecure
54     '';
55 };
56
57 # Use the certificates in services
58 services.nginx.virtualHosts."example.com" = {
59     sslCertificate = config.clan.core.vars.generators.intermediate-cert.;
60     sslCertificateKey = config.clan.core.vars.generators.intermediate-key.;
61 };
62 }

```

Multi-Service Secret Sharing

Generate secrets that multiple services can use:

```

1 {
2   # Generate database credentials
3   clan.core.vars.generators.database = {
4     share = true; # Share across machines
5     files."password" = { };
6     files."connection-string" = { };
7     prompts.dbname = {
8       description = "Database name";
9       type = "line";
10    };
11    script = ''
12      # Generate password
13      tr -dc 'A-Za-z0-9' < /dev/urandom | head -c 32 > $out/password
14
15      # Create connection string
16      echo "postgresql://app:$(cat $out/password)@localhost/$prompts/dbname"
17      > $out/connection-string
18    '';
19  };
20
21  # PostgreSQL uses the password
22  services.postgresql = {
23    enable = true;
24    initialScript = pkgs.writeText "init.sql" ''
25      CREATE USER app WITH PASSWORD '${
26        builtins.readFile config.clan.core.vars.generators.database.files."password"
27      }';
28    '';
29  };
30
31  # Application uses the connection string
32  systemd.services.myapp = {
33    serviceConfig.EnvironmentFile =
34      config.clan.core.vars.generators.database.files."connection-string"
35  };
36 }

```

SSH Host Keys with Certificates

Generate SSH host keys and sign them with a CA:

```

1 {
2   # SSH Certificate Authority (shared)

```

```

3  clan.core.vars.generators.ssh-ca = {
4      share = true;
5      files."ca" = { secret = true; deploy = false; };
6      files."ca.pub" = { secret = false; };
7      runtimeInputs = [ pkgs.openssh ];
8      script = ''
9          ssh-keygen -t ed25519 -N "" -f $out/ca
10         mv $out/ca.pub $out/ca.pub
11     '';
12 };
13
14 # Host-specific SSH keys
15 clan.core.vars.generators.ssh-host = {
16     files."ssh_host_ed25519_key" = {
17         secret = true;
18         owner = "root";
19         group = "root";
20         mode = "0600";
21     };
22     files."ssh_host_ed25519_key.pub" = { secret = false; };
23     files."ssh_host_ed25519_key-cert.pub" = { secret = false; };
24     dependencies = [ "ssh-ca" ];
25     runtimeInputs = [ pkgs.openssh ];
26     script = ''
27         # Generate host key
28         ssh-keygen -t ed25519 -N "" -f $out/ssh_host_ed25519_key
29
30         # Sign with CA
31         ssh-keygen -s $in/ssh-ca/ca \
32             -I "host:${config.networking.hostName}" \
33             -h \
34             -V -5m:+365d \
35             $out/ssh_host_ed25519_key.pub
36     '';
37 };
38
39 # Configure SSH to use the generated keys
40 services.openssh = {
41     hostKeys = [{
42         path = config.clan.core.vars.generators.ssh-host.files."ssh_host_ed25519_key.pub";
43         type = "ed25519";
44     }];
45 };
46 }

```

WireGuard Mesh Network

Create a WireGuard configuration with pre-shared keys:

```

1 {
2   # Generate WireGuard keys for this host
3   clan.core.vars.generators.wireguard = {
4     files."privatekey" = {
5       secret = true;
6       owner = "systemd-network";
7       mode = "0400";
8     };
9     files."publickey" = { secret = false; };
10    files."preshared" = { secret = true; };
11    runtimeInputs = [ pkgs.wireguard-tools ];
12    script = ''
13      # Generate key pair
14      wg genkey > $out/privatekey
15      wg pubkey < $out/privatekey > $out/publickey
16
17      # Generate pre-shared key
18      wg genpsk > $out/preshared
19    '';
20  };
21
22  # Configure WireGuard
23  networking.wireguard.interfaces.wg0 = {
24    privateKeyFile = config.clan.core.vars.generators.wireguard.files."privatekey";
25
26    peers = [{
27      publicKey = "peer-public-key-here";
28      presharedKeyFile = config.clan.core.vars.generators.wireguard.files."preshared";
29      allowedIPs = [ "10.0.0.2/32" ];
30    }];
31  };
32 }
```

Conditional Generation Based on Machine Role

Generate different secrets based on machine configuration:

```

1 {
```

```

2  clan.core.vars.generators = lib.mkMerge [
3    # All machines get basic auth
4    {
5      basic-auth = {
6        files."htpasswd" = { };
7        prompts.username = {
8          description = "Username for basic auth";
9          type = "line";
10       };
11       prompts.password = {
12         description = "Password for basic auth";
13         type = "hidden";
14       };
15       runtimeInputs = [ pkgs.apacheHttpd ];
16       script = ''
17         htpasswd -nbB "$prompts/username" "$prompts/password" > $out/h
18       '';
19     };
20   }
21
22   # Only servers get API tokens
23   (lib.mkIf config.services.myapi.enable {
24     api-tokens = {
25       files."admin-token" = { };
26       files."readonly-token" = { };
27       runtimeInputs = [ pkgs.openssl ];
28       script = ''
29         openssl rand -hex 32 > $out/admin-token
30         openssl rand -hex 16 > $out/readonly-token
31       '';
32     };
33   })
34 ];
35 }

```

Backup Encryption Keys

Generate and manage backup encryption keys:

```

1  {
2    clan.core.vars.generators.backup = {
3      share = true; # Same key for all backup sources
4      files."encryption.key" = {
5        secret = true;

```

```
6     deploy = true;
7 };
8 files."encryption.pub" = { secret = false; };
9 runtimeInputs = [ pkgs.age ];
10 script = ''
11     # Generate age key pair
12     age-keygen -o $out/encryption.key 2>/dev/null
13
14     # Extract public key
15     grep "public key:" $out/encryption.key | cut -d: -f2 | tr -d ' ' \
16     > $out/encryption.pub
17 '';
18 };
19
20 # Use in backup service
21 services.borgbackup.jobs.system = {
22     encryption = {
23         mode = "repokey-blake2";
24         passCommand = "cat ${config.clan.core.vars.generators.backup.files
25     };
26 };
27 }
```

Tips and Best Practices

1. **Use dependencies** to build complex multi-stage generations
2. **Share generators** when the same secret is needed across machines
3. **Set appropriate permissions** for service-specific secrets
4. **Use prompts** for user-specific values that shouldn't be generated
5. **Combine secret and non-secret files** in the same generator when they're related
6. **Use conditional generation** with `lib.mkIf` for role-specific secrets

← Previous

Understanding Clan Vars - Concepts & Architecture

Next →

Troubleshooting Vars

Need help? Reach out to [our community](#).